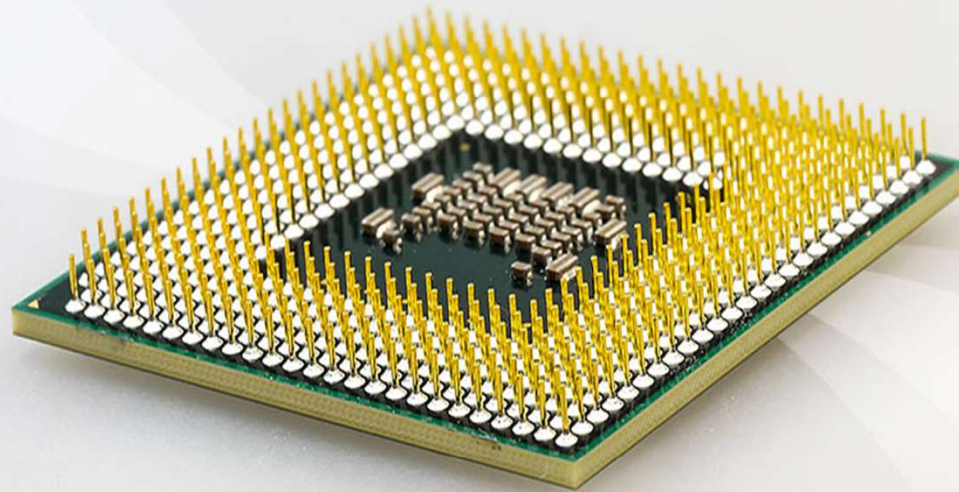


Đại Học Công Nghiệp TP HCM



THIẾT KẾ MẠCH TÍCH HỢP VỚI HDL

GV: Vũ Thị Hồng Nga

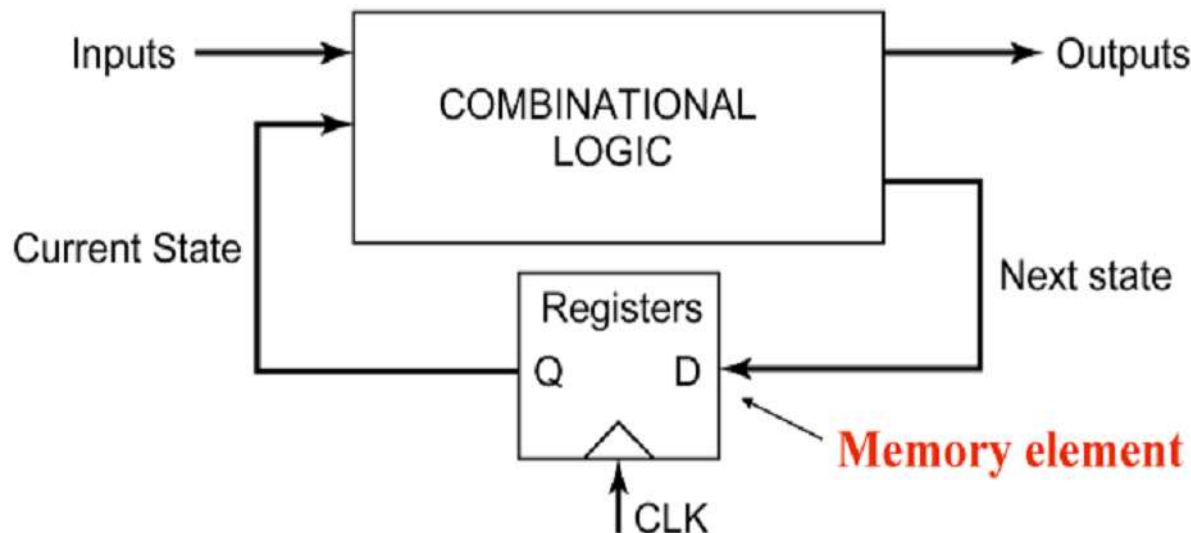
Chương 4: Thiết kế mạch tuần tự với Verilog HDL

Nội dung:

Mô tả các mạch tuần tự sau dùng ngôn ngữ verilog

- Flip Flop
- Thanh ghi dịch
- Mạch đếm nhị phân
- Mạch đếm BCD
- Máy trạng thái
- ROM/RAM
- Các mạch số ứng dụng

1. Đặc Điểm Chung Của Mạch Tuần Tự

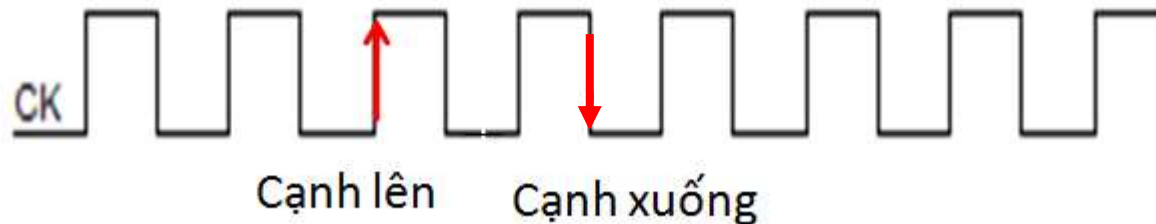


- Mạch tuần tự là mạch logic có tính chất nhớ, có khâu trễ
- Trạng thái tiếp theo của mạch phụ thuộc vào giá trị của kích thích ở ngõ vào và trạng thái hiện tại của chính nó.
- Mạch tuần tự thường hoạt động đồng bộ theo sự điều khiển của tín hiệu xung nhịp clock



1. Đặc Điểm Chung Của Mạch Tuần Tự (tt)

- Phần tử nhớ của mạch tuần tự thường là Flip Flop
- Xung nhịp (xung clock) là chuỗi xung vuông có biên độ và tần số xác định



- Mạch tuần tự thường có chân reset để thiết lập trạng thái ban đầu khi mạch hoạt động. Chân reset có thể đồng bộ hoặc không đồng bộ với xung nhịp ck.



Các dạng thường gặp khi lập trình ck và rs

ck tác động theo cạnh lên, rs không đồng bộ và tích cực thấp

always @ (posedge ck or negedge rs)

begin

if (rs == 1'b0)

...

end

ck tác động theo cạnh lên, rs không đồng bộ và tích cực cao

always @ (posedge ck or posedge rs)

begin

if (rs == 1'b1)

...

end



ck tác động theo cạnh lên, rs đồng bộ và tích cực thấp

```
always @ (posedge ck)
```

```
begin
```

```
if (rs == 1'b0)
```

```
...
```

```
end
```

ck tác động theo cạnh lên, rs đồng bộ và tích cực cao

```
always @ (posedge ck)
```

```
begin
```

```
if (rs == 1'b1)
```

```
...
```

```
end
```



ck tác động theo cạnh xuống, rs không đồng bộ và tích cực thấp

always @ (negedge ck or negedge rs)

begin

if (rs == 1'b0)

...

end

ck tác động theo cạnh xuống, rs không đồng bộ và tích cực cao

always @ (negedge ck or posedge rs)

begin

if (rs == 1'b1)

...

end



ck tác động theo cạnh xuống, rs đồng bộ và tích cực thấp

```
always @ (negedge ck)
begin
if (rs == 1'b0)
...
end
```

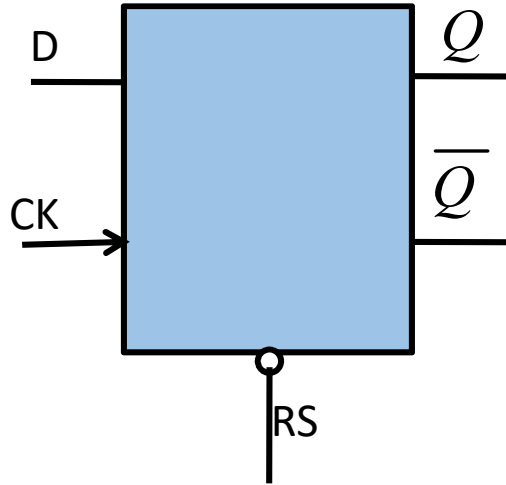
ck tác động theo cạnh xuống, rs đồng bộ và tích cực cao

```
always @ (negedge ck)
begin
if (rs == 1'b1)
...
end
```



2. Mô tả Flip Flop

Ví dụ : viết chương trình verilog mô tả D FF, ck tác động cạnh lên, rs không đồng bộ tích cực thấp



D _n	Q _{n+1}
0	0
1	1

```
module d_ff (ck, d, rs, q);  
  input ck, d, rs;  
  output q;  
  reg q;  
  always @ (posedge ck or negedge rs)  
  begin  
    if (rs == 1'b0)  
      q <= 0;  
    else  
      q <= d;  
  end  
end  
endmodule
```

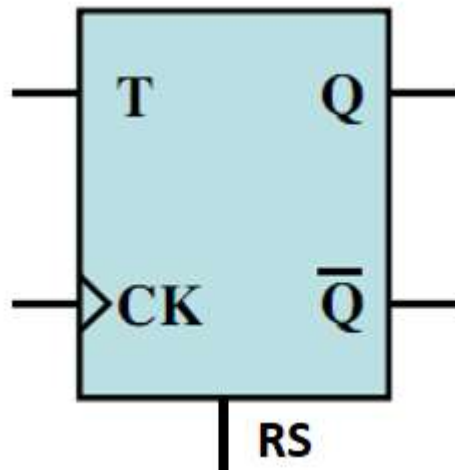


Ví dụ : viết chương trình verilog mô tả D_FF có ck tác động theo cạnh xuống, chân rs đồng bộ tích cực mức cao

```
module d_ff (ck, d, rs, q);  
input ck, d, rs;  
output q;  
reg q;  
always @ (negedge ck )  
begin  
    if (rs == 1'b1)  
        q <= 0;  
    else  
        q <= d;  
end  
endmodule
```



Ví dụ : viết chương trình verilog mô tả T_FF có ck tác động theo cạnh lên, chân rs đồng bộ tích cực mức cao



T_n	Q_{n+1}
0	Q_n
1	$\overline{Q_n}$



Chương trình verilog mô tả T_FF có ck tác động theo cạnh lên, chân rs đồng bộ tích cực mức cao

```
module t_ff (ck, t, rs, q);  
input ck, t, rs;  
output q;  
reg q;  
always @ (posedge ck )  
begin  
    if (rs == 1'b1)  
        q <= 0;  
    else  
        begin  
            if (t == 1'b0) q <= q;  
            else q <= ~q;  
        end  
    end  
end  
endmodule
```



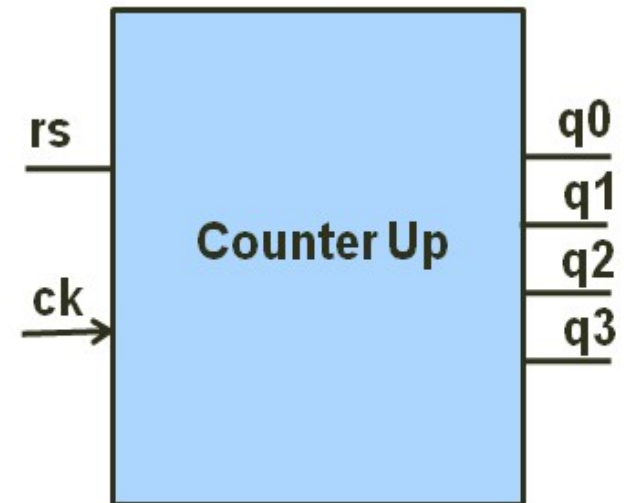
4. Mô tả bộ đếm nhị phân

- Bộ đếm nhị phân là bộ đếm có ngõ ra biểu diễn theo qui luật mã nhị phân
- Mã nhị phân n bit biểu diễn số nhị phân từ 0 đến $2^n - 1$
- Bộ đếm nhị phân có hai dạng là đếm lên hoặc đếm xuống
- Nếu bộ đếm nhị phân có $\text{mod} = 2^n$ thì ta chọn số ngõ ra là n bit $\rightarrow$ viết code cho bộ đếm nhị phân n bit
- Nếu bộ đếm nhị phân có mod khác 2^n thì ta chọn số ngõ ra là n bit sao cho $2^{n-1} < \text{mod} < 2^n \rightarrow$ viết code cho bộ đếm nhị phân n bit và xóa đi các trạng thái không mong muốn



Ví dụ : viết chương trình verilog mô tả bộ đếm lên nhị phân 4 bit, ck tác động theo cạnh lên, rs không đồng bộ tích cực mức cao

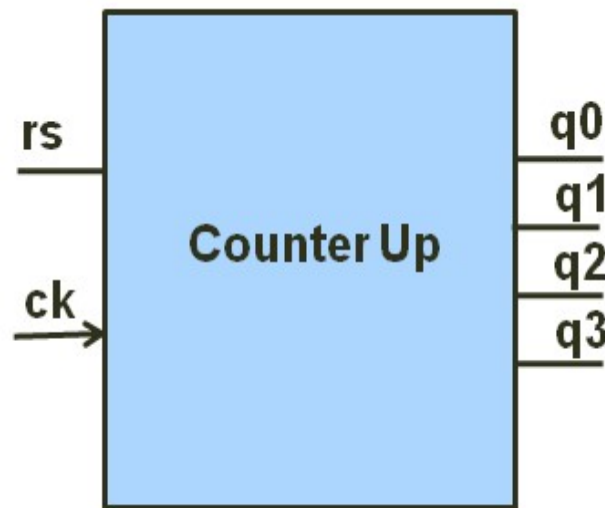
```
module counter_4bit (ck,rs,q);  
input ck, rs;  
output [3:0]q;  
reg [3:0]q;  
always @ (posedge ck or posedge rs)  
begin  
    if (rs == 1)  
        q <= 4'b 0000;  
    else q <= q + 1;  
end  
endmodule
```



Ví dụ: viết chương trình verilog mô tả mạch **đếm lên nhị phân 00-12**, ck tác động theo cạnh lên, rs không đồng bộ tích cực mức cao

Giải :

Mạch đếm có $\text{mod} = 13 \rightarrow$ chọn $n = 4 \rightarrow$ viết code mô tả mạch đếm nhị phân 4 bit và xóa đi các trạng thái > 12



Đáp án : chương trình verilog mô tả bộ đếm lên nhị phân 0-12, xung ck kích theo cạnh lên, reset tích cực mức cao

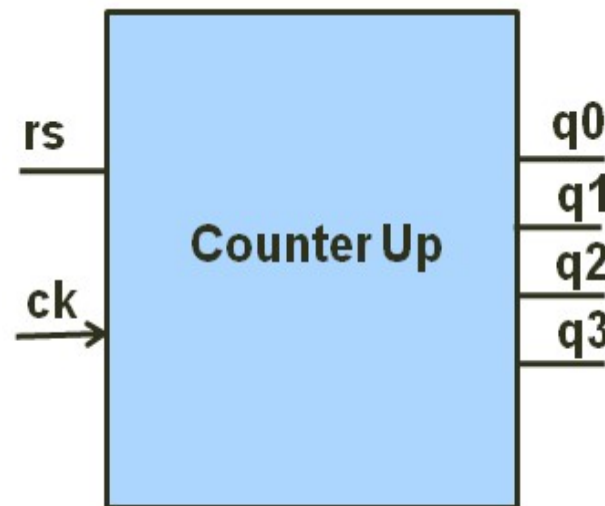
```
module counter (rs, ck, out);  
  input ck,rs;  
  output [3:0] out;  
  reg [3:0] out;  
  always @ (posedge ck or posedge rs) begin  
    if (rs == 1) out <= 4'b0000;  
    else  
      begin  
        out <= out +1;  
        if (out == 4'b1100) out <= 4'b0000;  
      end  
    end  
end  
endmodule
```



Ví dụ: viết chương trình verilog mô tả mạch **đếm xuống nhị phân 13 $\rightarrow$ 0**, ck tác động theo cạnh lên, rs không đồng bộ tích cực mức cao

Giải :

Mạch đếm có $\text{mod} = 14 \rightarrow$ chọn $n = 4 \rightarrow$ viết code mô tả mạch đếm nhị phân 4 bit và xóa đi các trạng thái >13



Đáp án : chương trình verilog mô tả bộ đếm xuống nhị phân 13→0, xung ck kích theo cạnh lên, reset tích cực mức cao

```
module counter (rs, ck, out);  
  input ck,rs;  
  output reg [3:0] out = 4'b1101;  
  always @ (posedge ck or posedge rs) begin  
    if (rs == 1) out <= 4'b1101;  
    else  
      begin  
        out <= out -1;  
        if (out == 4'b0000) out <= 4'b1101;  
      end  
  end  
end  
endmodule
```



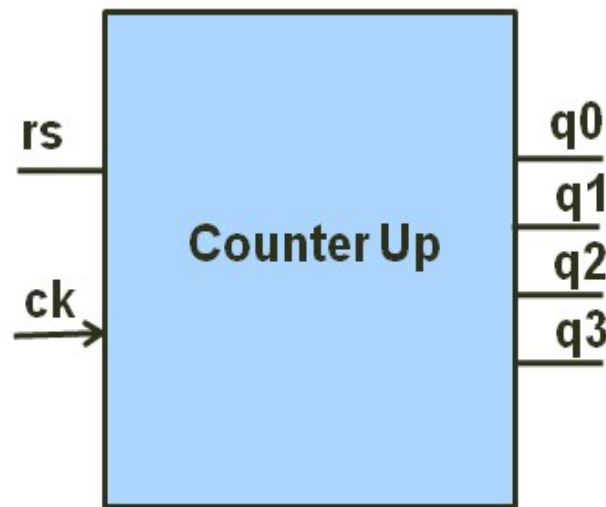
5. Mô tả bộ đếm BCD

- Bộ đếm BCD là bộ đếm có ngõ ra biểu diễn theo qui luật mã BCD
- Mỗi số BCD được biểu diễn bằng 4 bit nhị phân từ 0000 đến 1001.
- Mã BCD 1 digit biểu diễn số thập phân từ 0 đến 9
- Mã BCD 2 digit biểu diễn số thập phân từ 00 đến 99
- Mã BCD 3 digit biểu diễn số thập phân từ 000 đến 999
-
- Mô tả bộ đếm BCD 1 digit sẽ có 4 ngõ ra
- Mô tả bộ đếm BCD 2 digit sẽ có 8 ngõ ra
- Mô tả bộ đếm BCD n digit sẽ có $n*4$ ngõ ra



Ví dụ : Viết chương trình verilog mô tả bộ đếm lên BCD 1 digit, chân rs không đồng bộ tích cực mức cao, ck tác động theo cạnh lên.

Gợi ý: viết giống bộ đếm lên nhị phân từ 0 đến 9

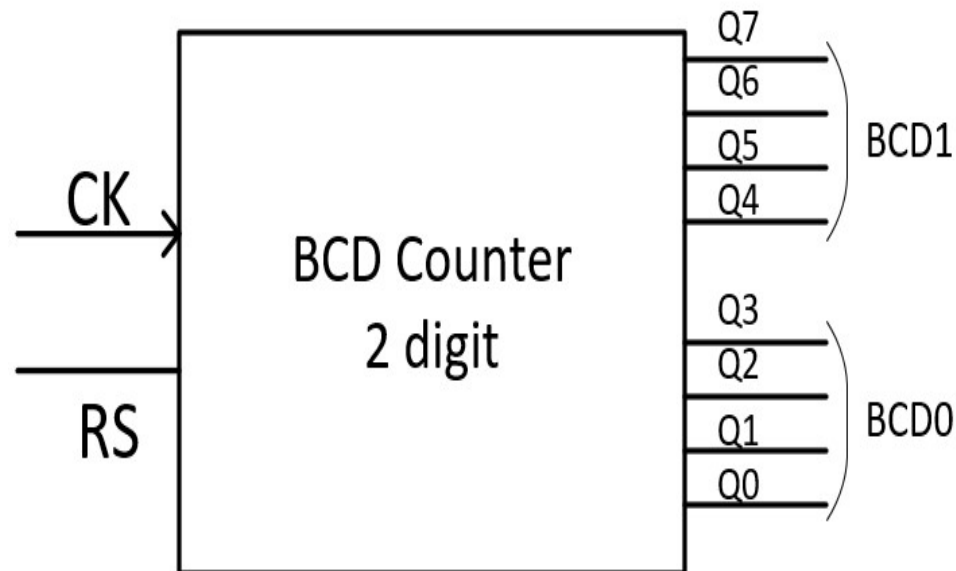


Đáp án : chương trình cho bộ đếm lên BCD 1 digit (đếm 0→9), xung ck kích theo cạnh lên, reset không đồng bộ tích cực mức cao

```
module counter (rs, ck, bcd);  
  input ck,rs;  
  output [3:0] bcd;  
  reg [3:0] bcd;  
  always @ (posedge ck or posedge rs) begin  
    if (rs == 1) bcd <= 4'b0000;  
    else  
      begin  
        bcd <= bcd +1;  
        if (bcd == 4'b1001) bcd <= 4'b0000;  
      end  
    end  
  end  
endmodule
```



Ví dụ : viết chương trình verilog mô tả bộ đếm lên BCD 2 digit, đếm lên từ 00 $\rightarrow$ 99, chân rs đồng bộ tích cực mức cao, ck tác động theo cạnh lên.



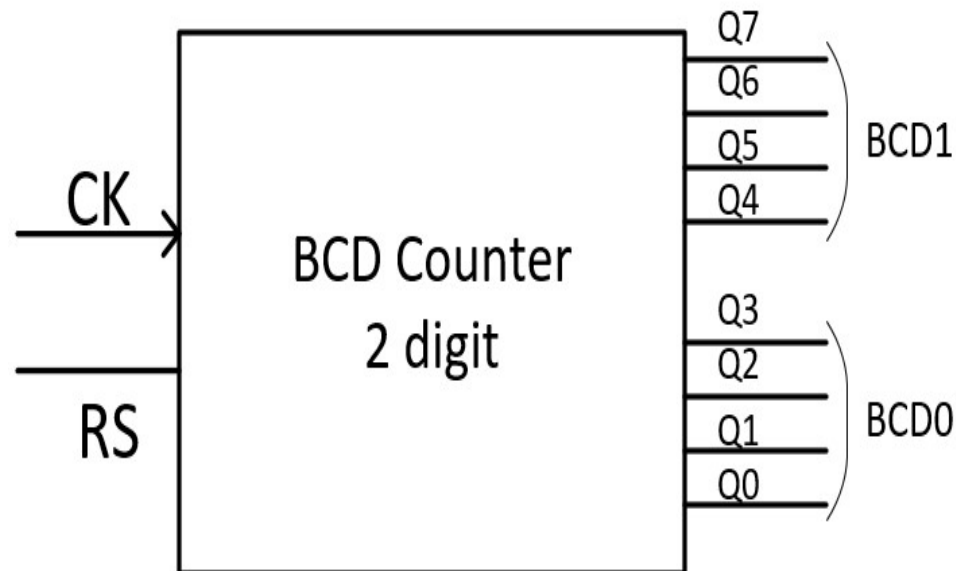
code mô tả mạch đếm BCD 2 digit đếm 00 → 99

```
//mạch đếm 00-99
//ngo ra ma BCD
//bcd1 : hàng đơn vị
//bcd0 : hàng chục
//-----//
module counter_bcd (ck,rs,bcd1,bcd0);
input ck, rs;
output [3:0] bcd1,bcd0;
reg [3:0] bcd1, bcd0;
```

```
always @ (posedge ck)
begin
    if (rs == 1)
    begin
        bcd1 <= 4'b0000;
        bcd0 <= 4'b0000;
    end
    else
    begin
        bcd0 <= bcd0 + 1;
        if (bcd0 == 4'b1001)
        begin
            bcd0 <= 4'b0000;
            bcd1 <= bcd1 + 1;
            if (bcd1 == 4'b1001)
                bcd1 <= 4'b0000;
        end
    end
end
endmodule
```



Ví dụ : viết chương trình verilog mô tả bộ đếm lên BCD 2 digit, đếm lên từ 00 $\rightarrow$ 12, chân rs đồng bộ tích cực mức cao, ck tác động theo cạnh lên.



```

module counter_bcd_00_12 (ck,rs,bcd1,bcd2);
input ck, rs;
output [3:0] bcd1,bcd2;
reg [3:0] bcd1, bcd2;
always @ (posedge ck)
begin
    if (rs == 1)
    begin
        bcd1 <= 4'b0000;
        bcd2 <= 4'b0000;
    end
    else
    begin
        bcd1 <= bcd1 + 1;
        if (bcd1 == 4'b1001)
        begin
            bcd1 <= 4'b0000;
            bcd2 <= bcd2 + 1;
        end
        if ((bcd2 == 4'b0001)&&(bcd1 == 4'b0010))
        begin
            bcd2 <= 4'b0000;
            bcd1 <= 4'b0000;
        end
    end
end
endmodule

```

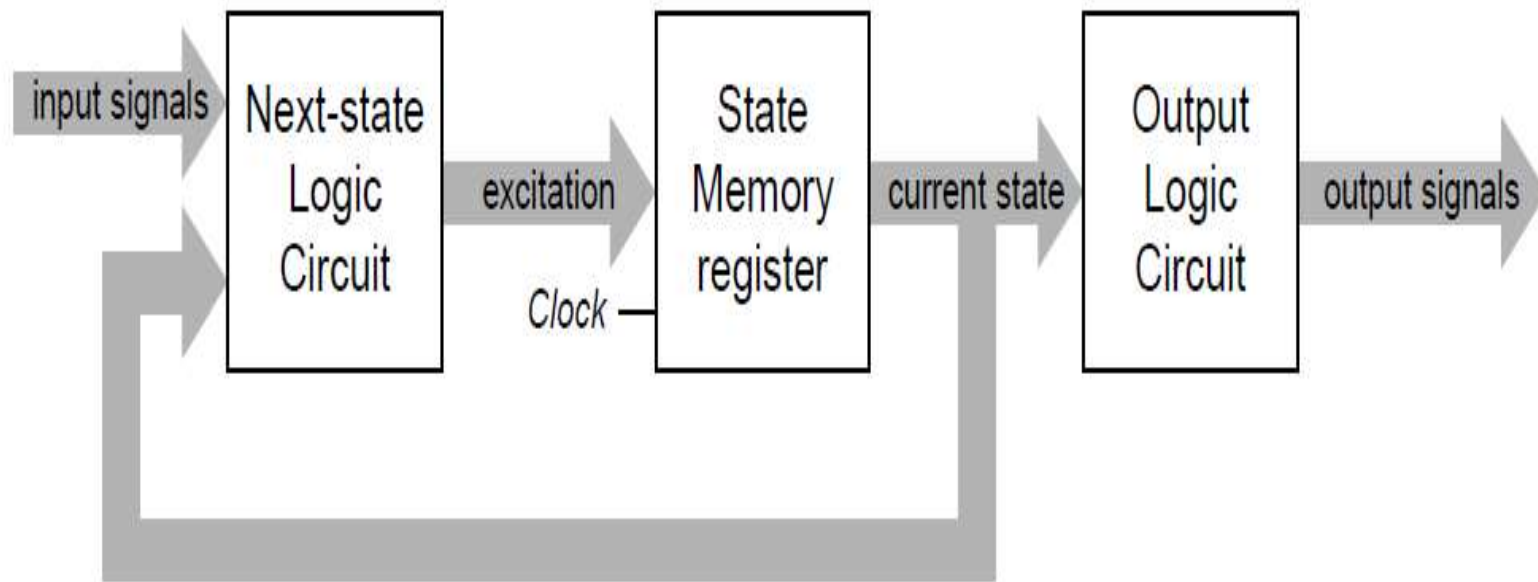
6. MÁY TRẠNG THÁI (Finite state machines -FSM)

a. Giới thiệu chung về FSM:

- FSM là một công cụ dùng để mô tả các hành vi của hệ thống số. Nó chỉ ra tất cả các trạng thái có thể có của hệ thống, và những sự kiện nào sẽ gây ra sự thay đổi trạng thái.
- Các ngõ ra của FSM phụ thuộc vào ngõ vào, xung ck và trạng thái nhớ của các flip flop bên trong FSM
- FSM thường được mô tả bằng giản đồ Moore hoặc Mealy



b. Sơ đồ khối FSM dạng moore



Next state logic circuit: khối tác động trạng thái kế tiếp

State memory register: khối thanh ghi (lưu trạng thái trong của FSM)

Output logic circuit: khối tác động ngõ ra



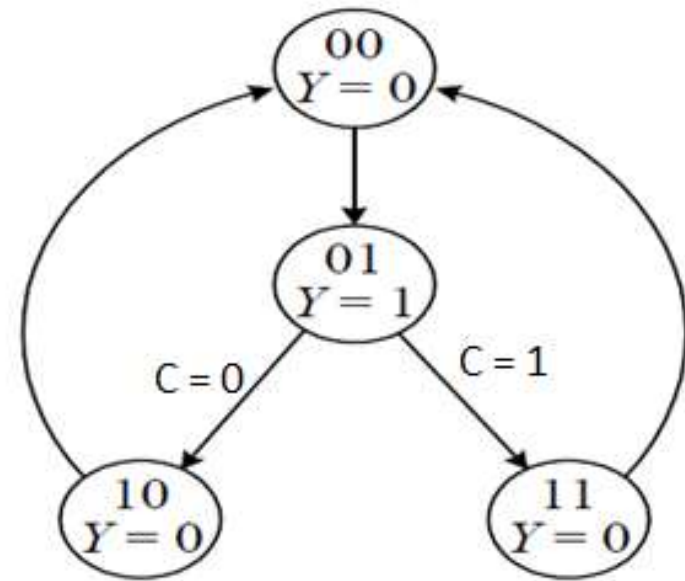
Ví dụ : FSM mô tả theo giản đồ trạng thái dạng Moore

Ngõ vào: C

Ngõ ra : Y

4 trạng thái trong: 00,01,10,11

Chiều mũi tên là chiều chuyển
trạng thái



Ta có thể mã hóa các trạng thái của FSM thành các nhãn mong muốn:

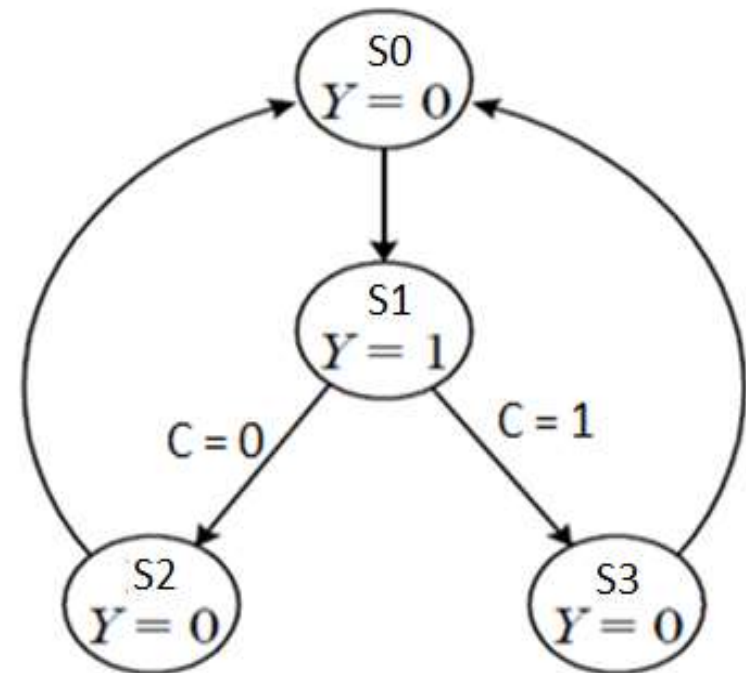
VD:

s0 là trạng thái 00

s1 là trạng thái 01

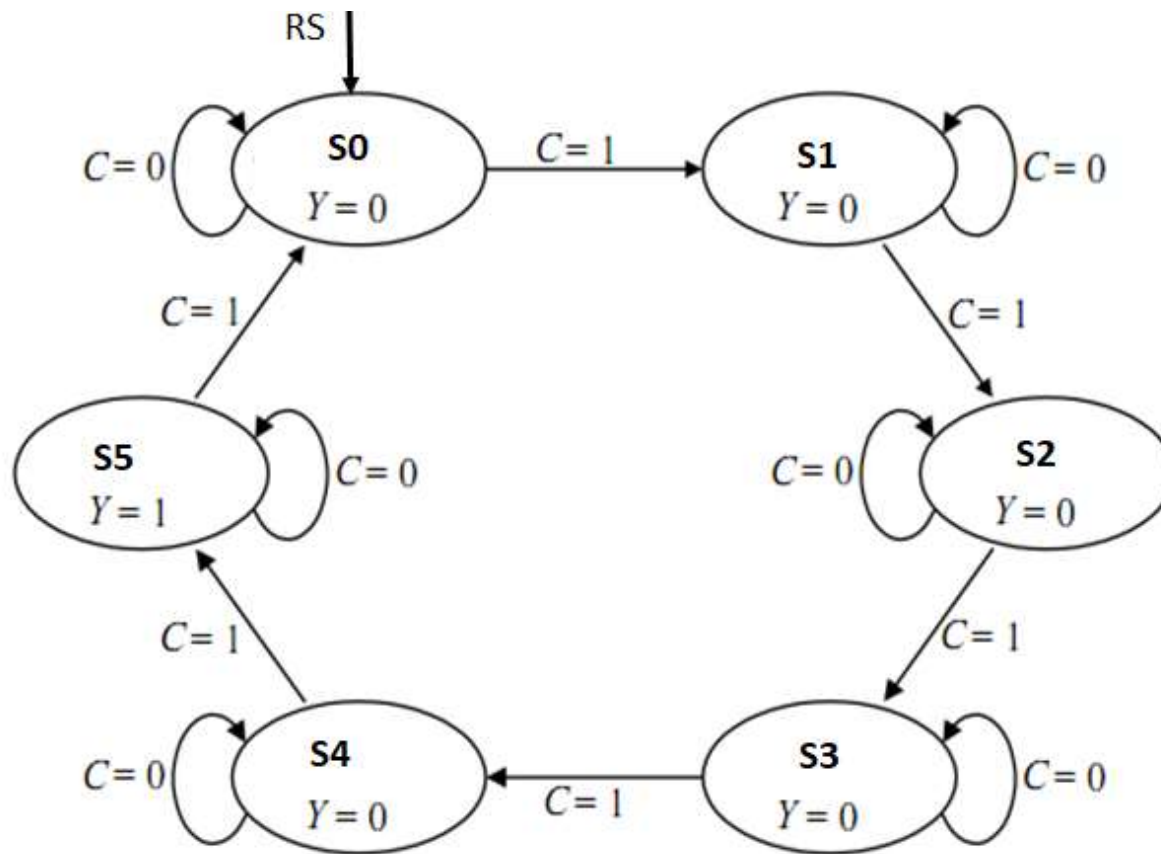
s2 là trạng thái 10

s3 là trạng thái 11

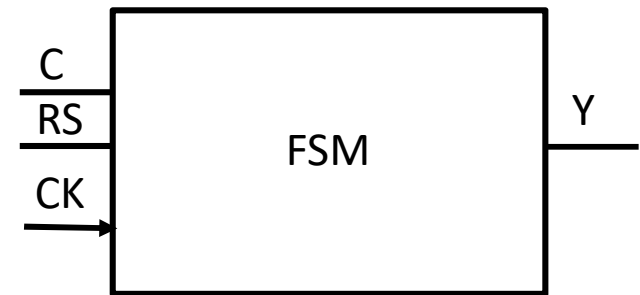


Ví dụ : Viết code Verilog mô tả máy trạng thái sau. chân rs đồng bộ tích cực mức cao, ck tác động theo cạnh lên.

Gráp trạng thái của FSM



Sơ đồ chân của FSM



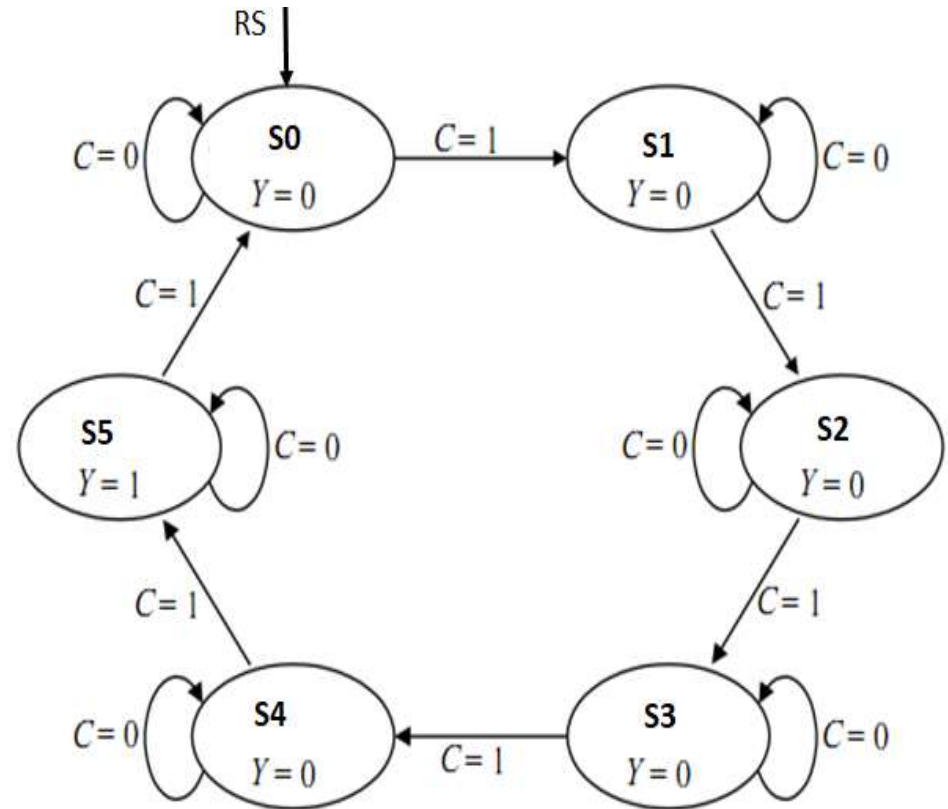
```
module fsm_VD1 (ck,rs,c,y);  
parameter s0 = 3'b000;  
parameter s1 = 3'b001;  
parameter s2 = 3'b010;  
parameter s3 = 3'b011;  
parameter s4 = 3'b100;  
parameter s5 = 3'b101;  
input ck,rs,c;  
output y;  
reg [2:0]current_state, next_state;  
reg y;
```

```
always @ (*)  
begin  
case (current_state)  
s0 : if (c) next_state = s1;  
     else next_state = current_state;  
s1 : if (c) next_state = s2;  
     else next_state = current_state;  
s2 : if (c) next_state = s3;  
     else next_state = current_state;  
s3 :   if (c) next_state = s4;  
       else next_state = current_state;  
s4 :   if (c) next_state = s5;  
       else next_state = current_state;  
s5 :   if (c) next_state = s0;  
       else next_state = current_state;  
default next_state = s0;  
endcase  
end
```

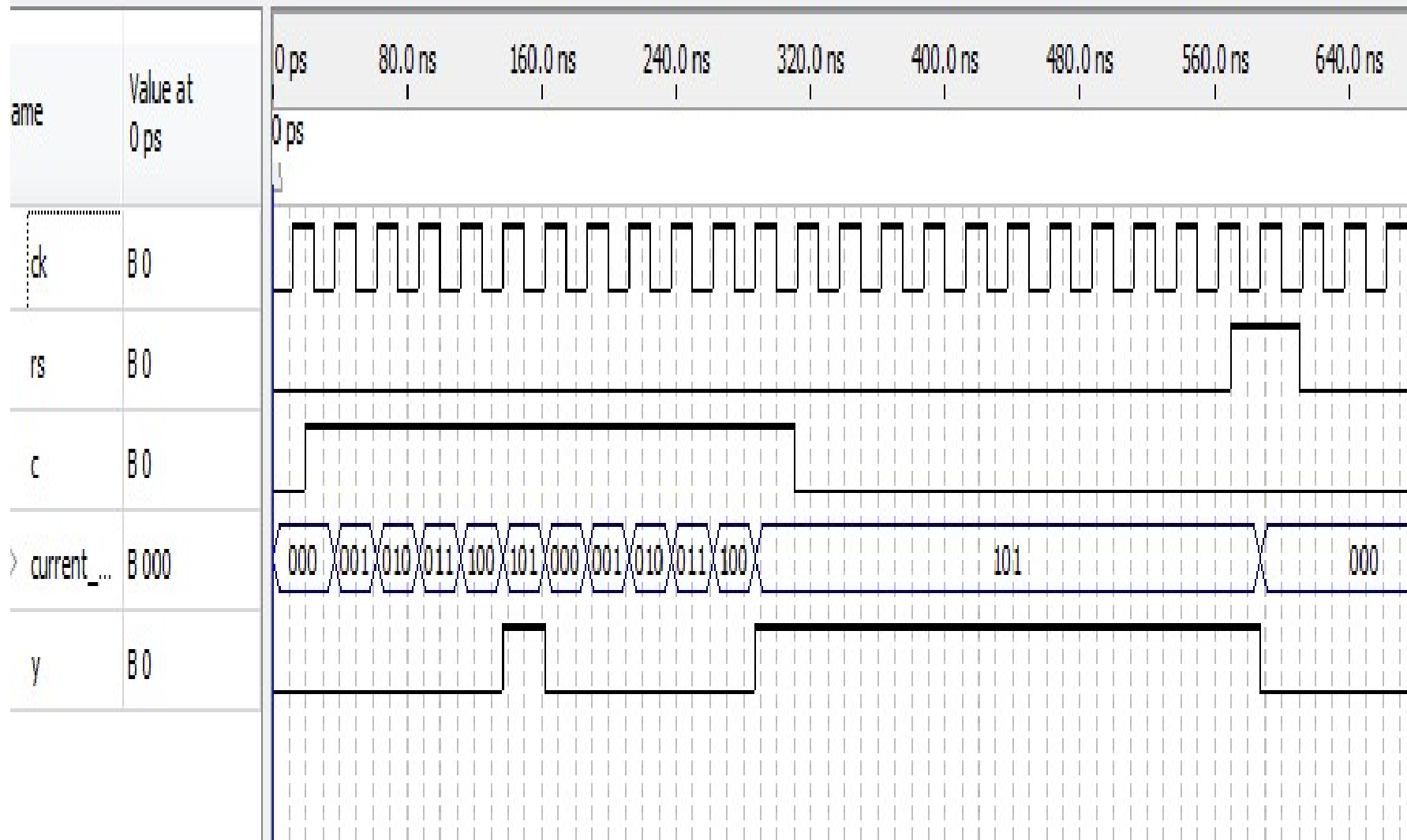


```
always @ (posedge ck)
begin
  if (rs == 1) current_state <= s0;
  else current_state <= next_state;
End
```

```
always @ (*)
begin
  if (current_state == s5) y = 1'b1;
  else y = 1'b0;
end
Endmodule
```

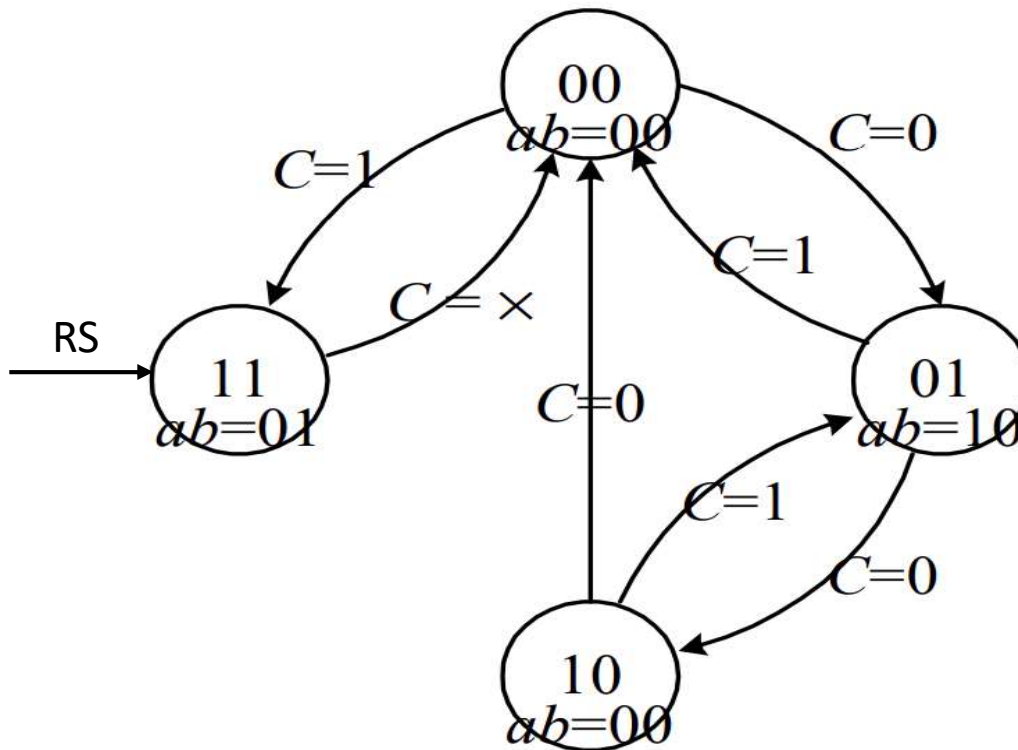


Kết quả mô phỏng



Ví dụ:

Viết code verilog mô tả FSM theo sơ đồ sau, chân rs không đồng bộ tích cực mức cao, ck tác động theo cạnh lên.



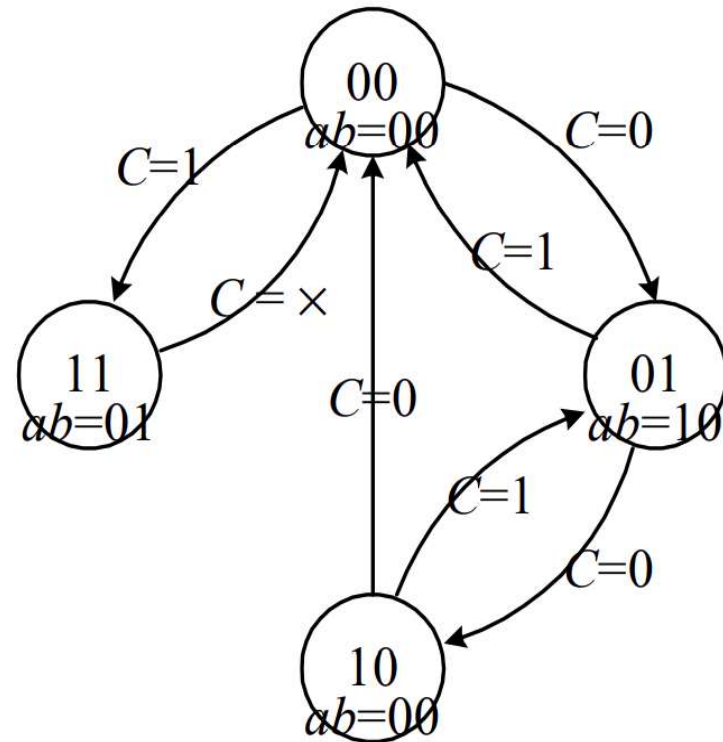
```
module fsm_VD2 (ck,rs,c,ab);  
parameter s0 = 2'b00;  
parameter s1 = 2'b01;  
parameter s2 = 2'b10;  
parameter s3 = 2'b11;  
input ck,rs,c;  
output [1:0]ab;  
reg [1:0]current_state, next_state;  
reg [1:0]ab;
```

```
always @ (*)  
begin  
case (current_state)  
s0 : if (c) next_state = s3;  
     else next_state = s1;  
s1 : if (c) next_state = s0;  
     else next_state = s2;  
s2 : if (c) next_state = s1;  
     else next_state = s0;  
s3 : next_state = s0;  
default next_state = s0;  
endcase  
end
```



```
always @ (posedge ck or posedge rs)
begin
  if (rs == 1) current_state <= s3;
  else current_state <= next_state;
end
```

```
always @ (*)
case (current_state)
  s0 : ab = 2'b00;
  s1 : ab = 2'b10;
  s2 : ab = 2'b00;
  s3 : ab = 2'b01;
endcase
endmodule
```

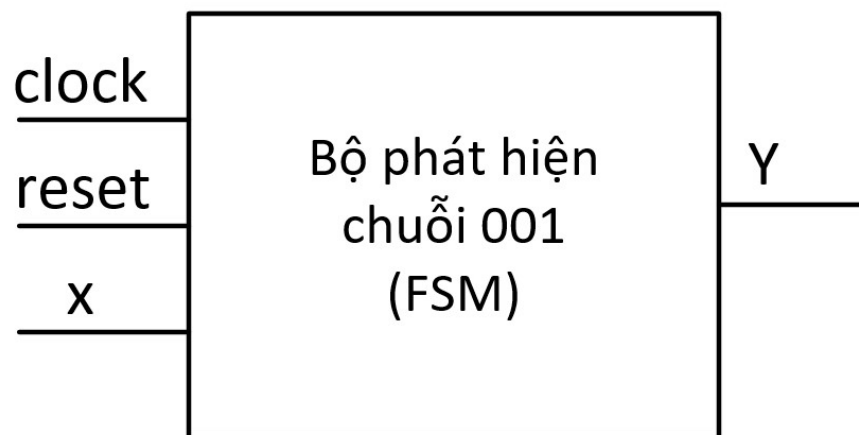


Ví dụ:

viết chương trình verilog mô tả FSM có chức năng phát hiện chuỗi 001 từ ngõ vào x, rs đồng bộ tích cực thấp, ck tác động theo cạnh lên. Khi phát hiện đúng chuỗi 001 ở ngõ vào thì ngõ ra bằng 1

- Vẽ grap trạng thái
- Viết code verilog

Mã hóa trạng thái:

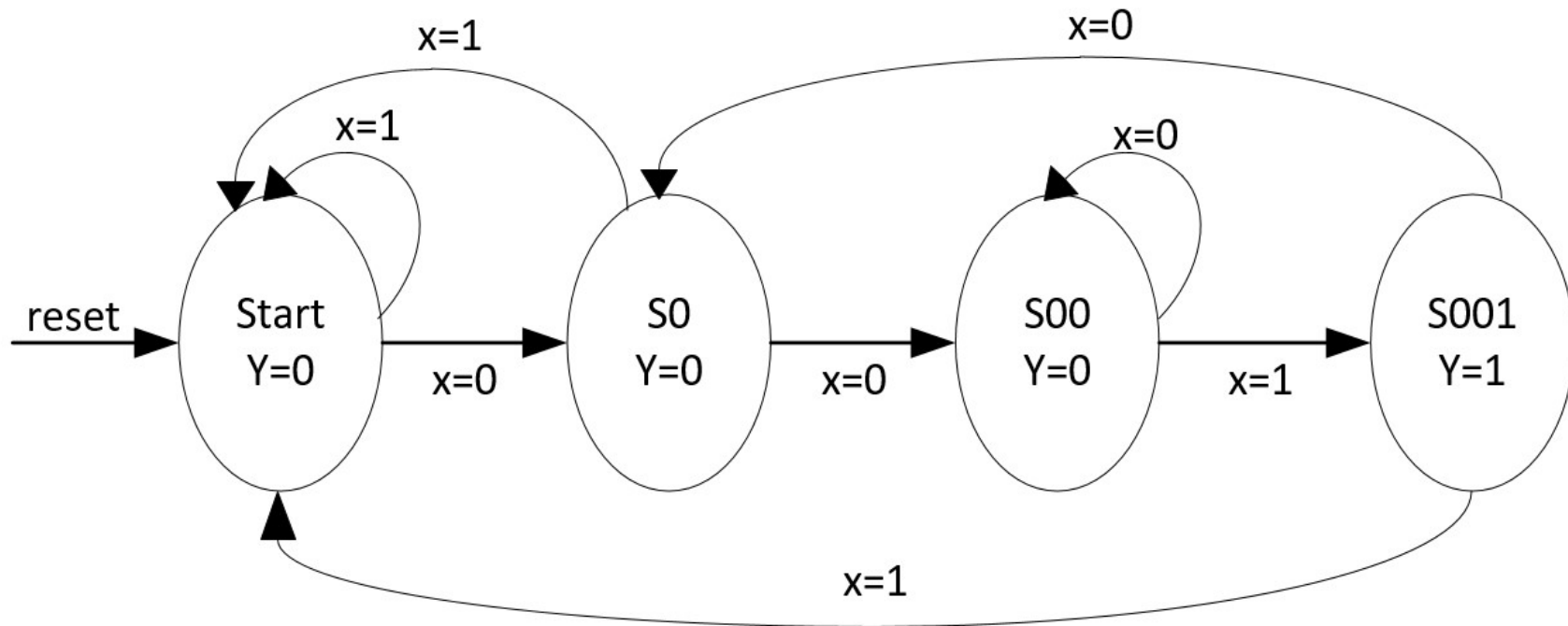


start	00
S0	01
S00	10
S001	11



Bài giải:

a. vẽ graph trạng thái



x: ngõ vào

y: ngõ ra

rs : reset tích cực mức thấp

Ck: tác động theo cạnh lên

start, s0, s00, s001 : là các trạng thái trong

Khi phát hiện ngõ vào là chuỗi số 001 thì ngõ ra bằng 1



//khai báo module, tham số, input, output

```
//phát hiện chuỗi 001 ở ngõ vào  
module fsm1(ck,rs,x,y);  
parameter start = 2'b00;  
parameter s0 = 2'b01;  
parameter s00 = 2'b10;  
parameter s001 = 2'b11;  
input ck, rs,x;  
output reg y;  
reg [1:0] next_state, current_state;
```



// mạch tổ hợp xác định trạng thái kế tiếp

```
always @ (*)
begin
  case (current_state)
  start:
    begin
      if (!x) next_state = s0;
      else next_state = current_state;
    end
  s0:
    begin
      if (!x) next_state = s00;
      else next_state = start;
    end
  s00:
    begin
      if (x) next_state = s001;
      else next_state = current_state;
    end
  end
```

```
s001:
  begin
    if (x) next_state = start;
    else next_state = s0;
  end
  default: next_state = start;
endcase
End
```



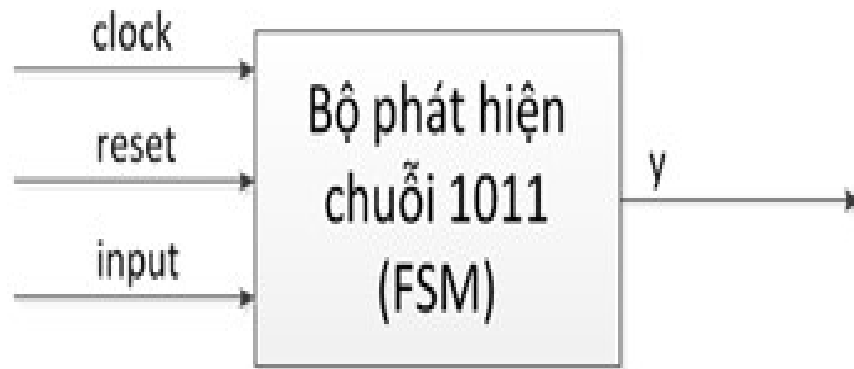
// thanh ghi trạng thái trong và ngõ ra của FF

```
always @(posedge ck)
begin
    if (!rs) current_state <= start;
        else current_state <= next_state;
end
//mạch xác định trạng thái của ngõ ra
always @ (*)
begin
    if (current_state == s001) y = 1'b1;
        else y = 1'b0;
end
endmodule
```

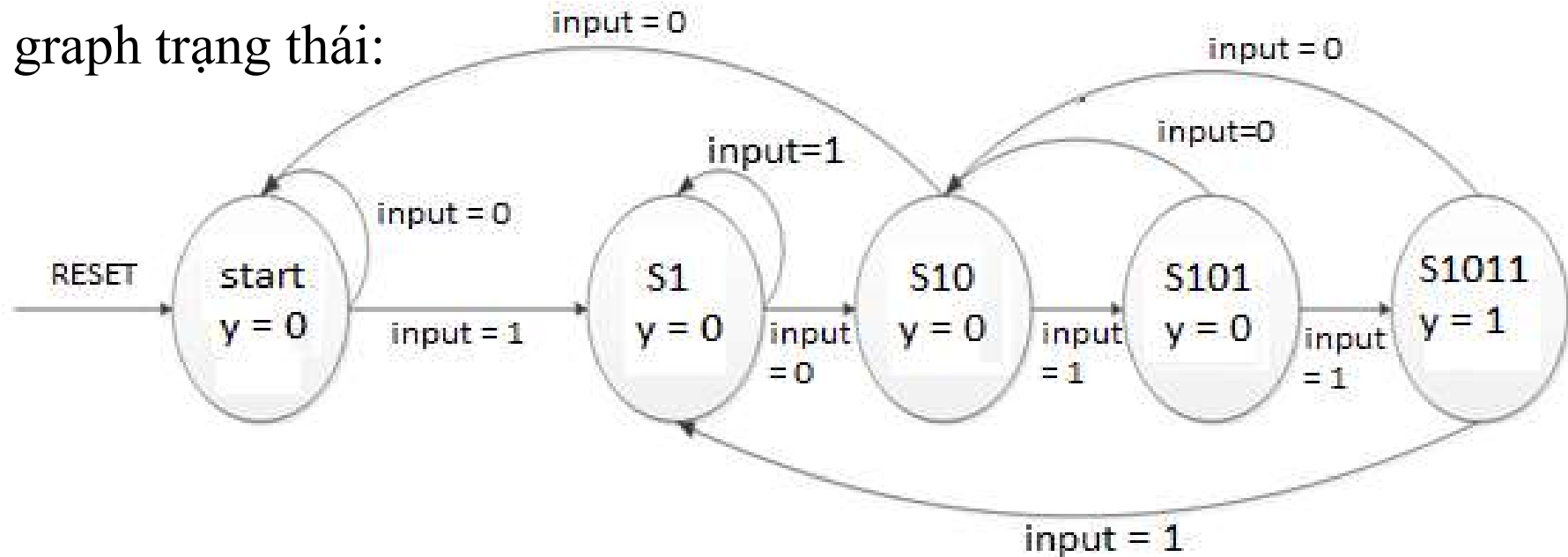


Ví dụ: viết chương trình verilog mô tả FSM có chức năng phát hiện chuỗi 1011 từ ngõ vào. Chân ck kích theo cạnh lên, rs đồng bộ tích cực mức cao. Ngõ ra sẽ bằng 1 khi phát hiện đúng chuỗi 1011 ở ngõ vào

Mã hóa trạng thái:



start	000
s1	001
s10	010
s101	011
s1011	100

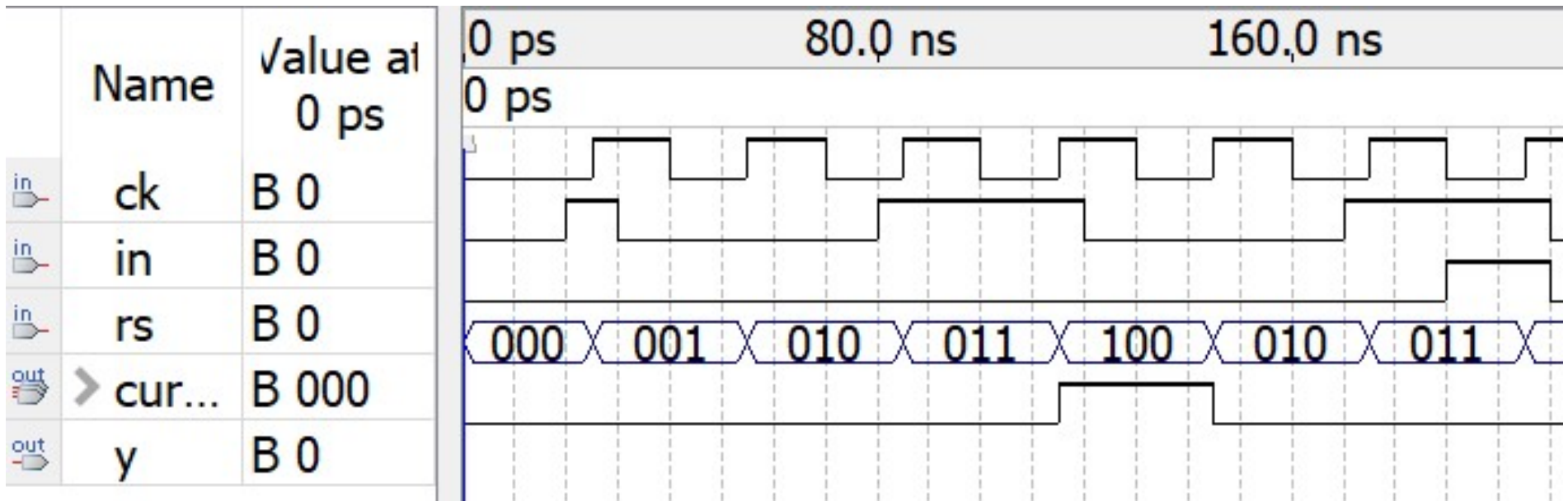


```

module fsm_bt2 (ck,rs,in,y);
parameter start      = 3'b000;
parameter s1         = 3'b001;
parameter s10        = 3'b010;
parameter s101       = 3'b011;
parameter s1011      = 3'b100;
input ck,rs,in;
output reg y;
reg [2:0]current_state, next_state;
always @ (*)
begin
    case (current_state)
    start :      if (in) next_state = s1;
                  else next_state = current_state;
    s1      :  if (~in) next_state = s10;
                  else next_state = current_state;
    s10     :      if (in) next_state = s101;
                  else next_state = start;
    s101    :      if (in) next_state = s1011;
                  else next_state = s10;
    s1011   :      if (in) next_state = s1;
                  else next_state = s10;
    default next_state = start;
    endcase
end
always @ (posedge ck)
begin
    if (rs) current_state <= start;
    else current_state <= next_state;
end
always @ (*)
begin
    if (current_state == s1011) y = 1'b1;
    else y = 1'b0;
end
endmodule

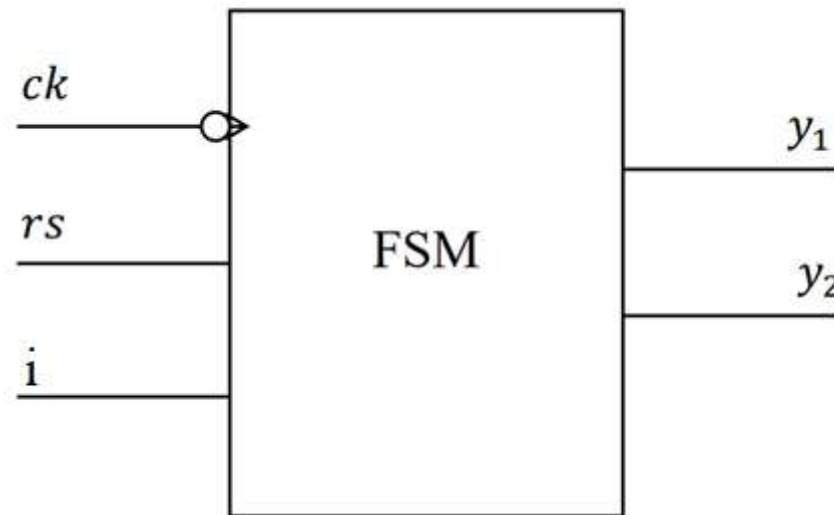
```

Kết quả mô phỏng

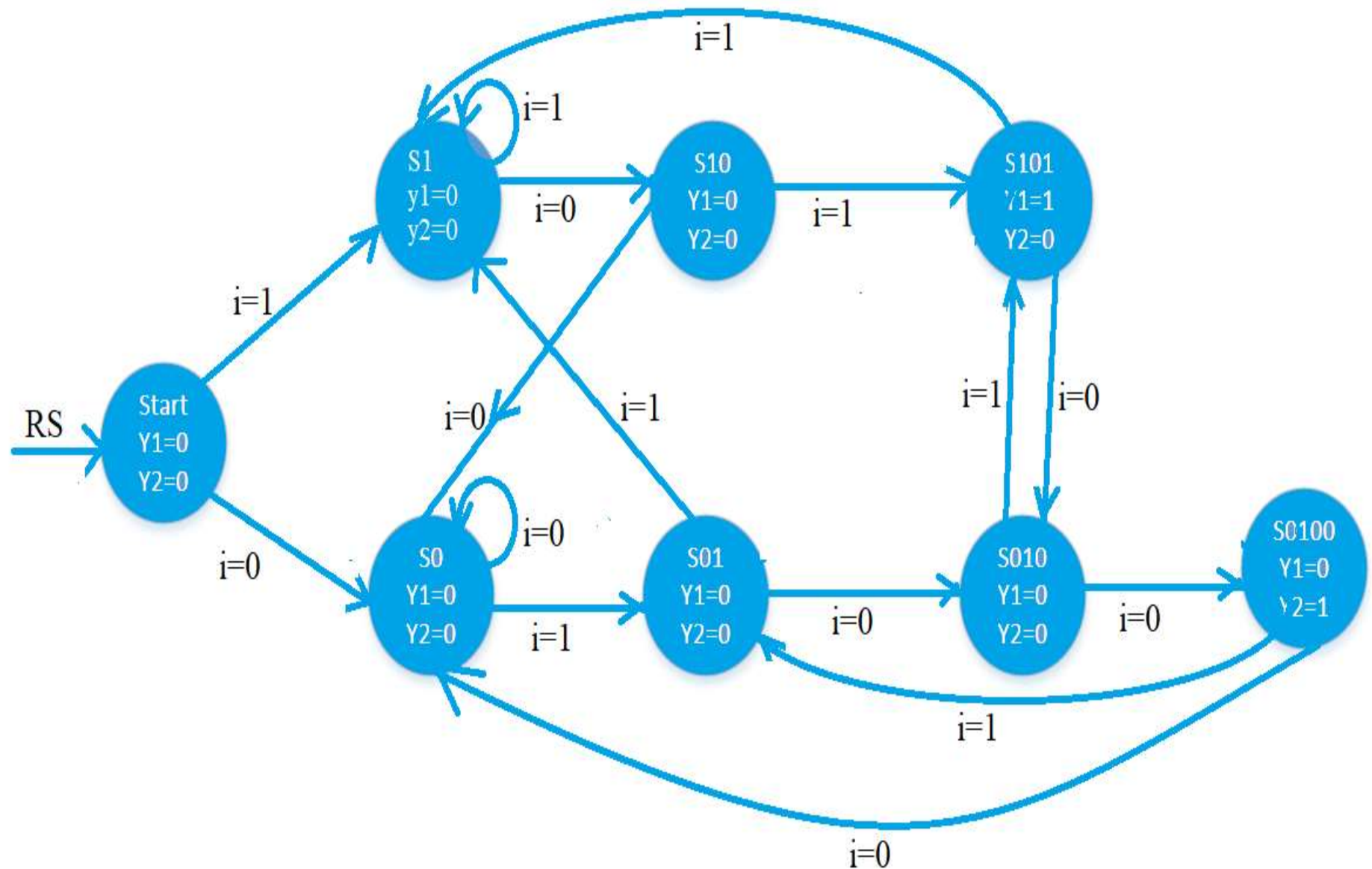


Bài tập:

Viết chương trình verilog mô tả máy trạng thái phát hiện chuỗi bit 101 và 0100 ở ngõ vào. Ngõ ra $Y1=1$ khi phát hiện đúng chuỗi 101, $Y2=1$ khi phát hiện đúng chuỗi 0100. Xung ck tác động theo cạnh xuống, rs không đồng bộ tích cực mức cao

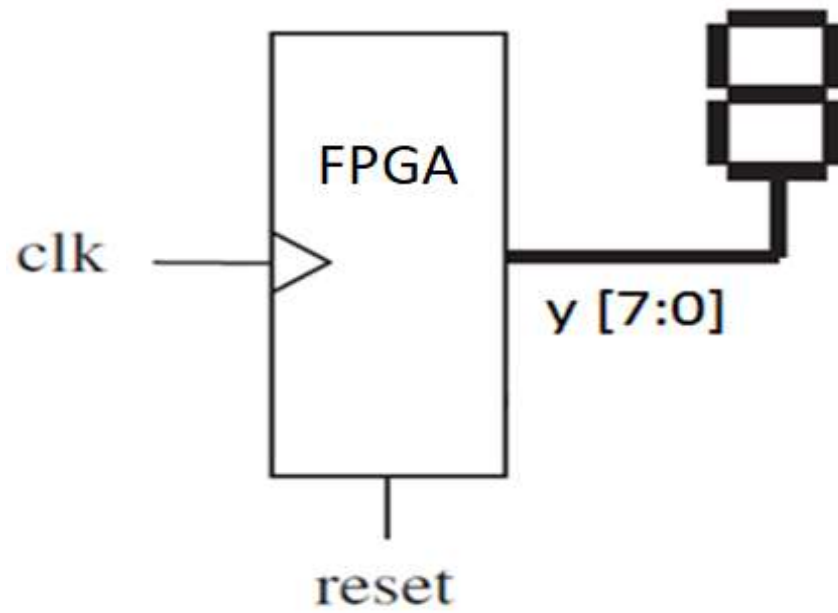


Gợi ý: viết code theo graph trạng thái sau



6. Lập trình mô tả mạch ứng dụng

Ví dụ: lập trình verilog điều khiển led bảy đoạn hiển thị số đếm lên $0 \rightarrow 9$, led 7 SEG loại anode chung, chân rs đồng bộ tích cực mức cao, thời gian hiển thị 1 trạng thái là 1s.

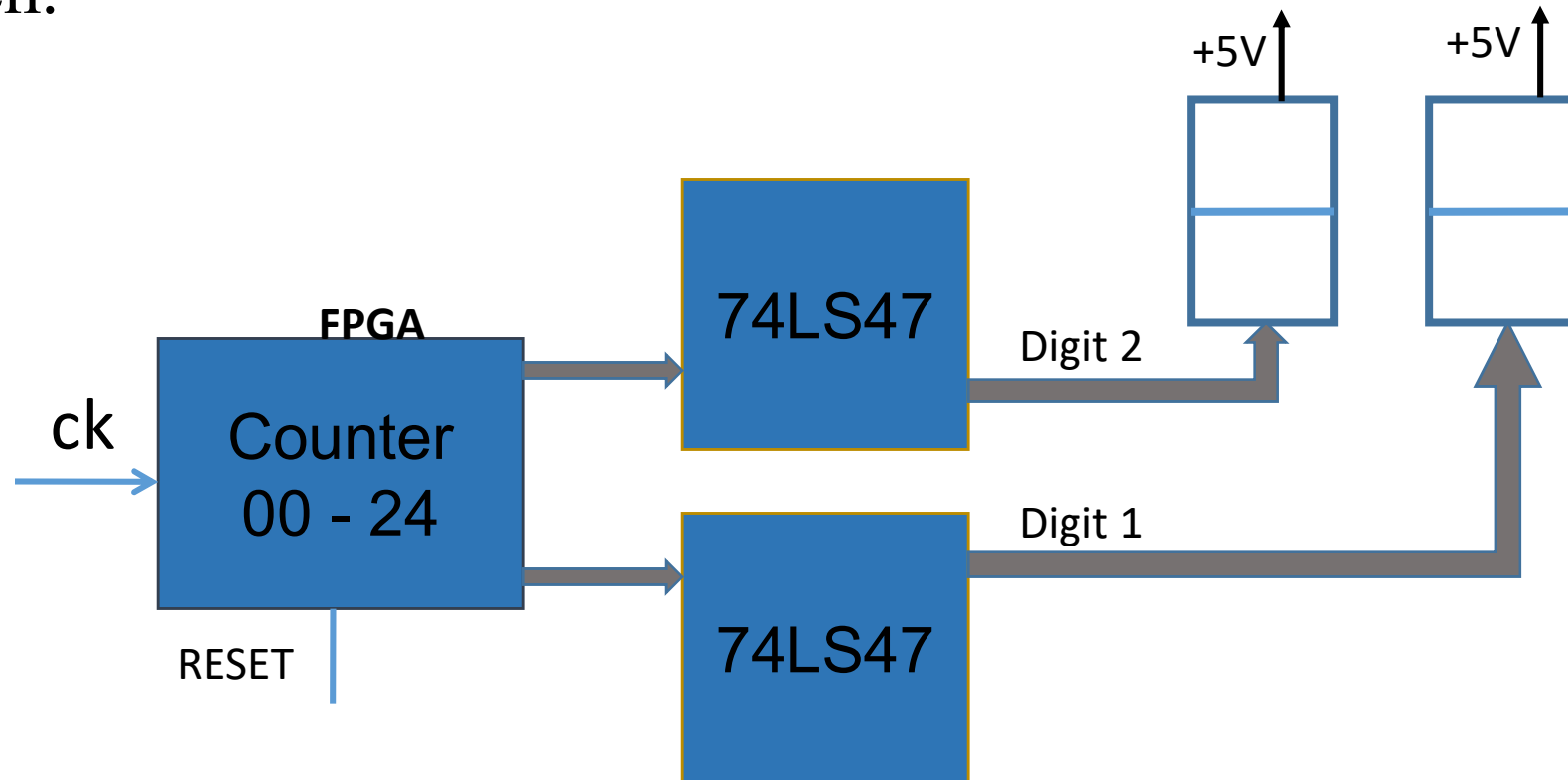


```
//mach dem 0-9, giai ma led 7 seg
module counter_giaima (ck,rs,led);
input ck, rs;
output [7:0]led;
reg [7:0]led;
reg [3:0]q;
always @ (posedge ck )
begin
    if (rs == 1) q <= 4'b 0000;
    else
    begin
        q <= q + 1;
        if (q == 4'b1001)
            q <= 4'b0000;
        end
    end
end
```

```
// giai ma led 7 seg
always @ (posedge ck)
begin
    always @ (q)
    begin
        case (q)
            4'b0000: display= 7'b 1000000; //0
            4'b0001: display= 7'b 1111001; //1
            4'b0010: display= 7'b 0100100; //2
            4'b0011: display= 7'b 0110000; //3
            4'b0100: display = 7'b 0011001; //4
            4'b0101: display = 7'b 0010010; //5
            4'b0110: display = 7'b 0000010; //6
            4'b0111: display = 7'b 1111000; //7
            4'b1000: display = 7'b 0000000; //8
            4'b1001: display = 7'b 0010000; //9
            default: display = 7'b 1111111; //off
        endcase
    end
end
endmodule
```



Ví dụ : viết chương trình verilog điều khiển 2 led đếm lên 00 đến 24, chân rs tích cực mức cao, ck tác động theo cạnh lên.



Đáp án : code mô tả mạch đếm 2 digit đếm 00 → 24

```
module BT (ck,rs,q,bcd1,bcd0);
input ck, rs;
output [4:0]q;
output [3:0]bcd1,bcd0;
reg [4:0]q;
reg [3:0]bcd1,bcd0;
always @ (posedge ck or posedge rs)
begin
    if (rs == 1) q <= 5'b 00000;
    else
    begin
        q <= q + 1;
        if (q == 5'b11000)
            q<= 5'b00000;
        end
    end
end
always @ (posedge ck or posedge rs)
begin
    bcd1 = q / 10;
    bcd0 = q % 10;
end
endmodule
```



3. Mô tả thanh ghi dịch:

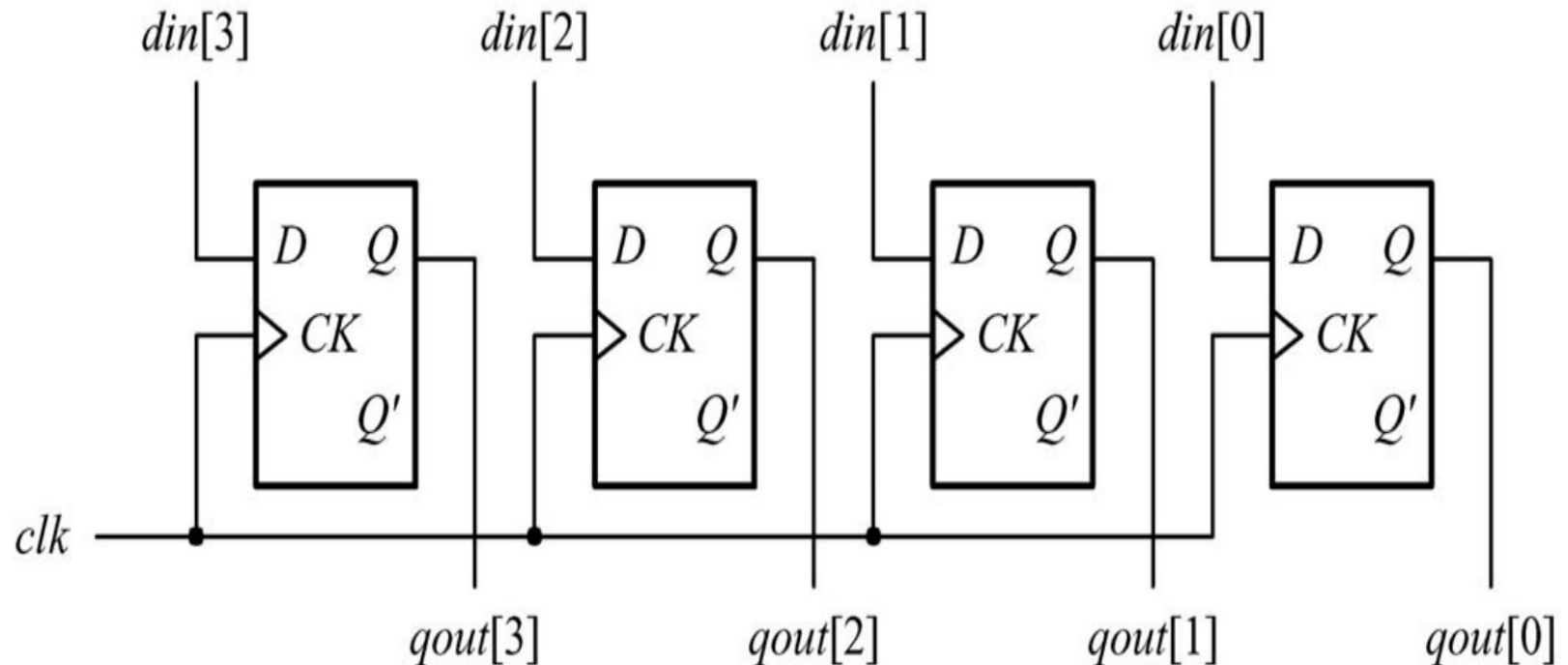
Các loại thanh ghi dịch thường gặp:

- SISO (serial in serial out)
- SIPO (serial in parallel out)
- PISO (parallel in serial out)
- PIPO (parallel in parallel out)



a. Thanh ghi dịch vào song song, ra song song (PIPO)

Ví dụ : viết chương trình verilog mô tả thanh ghi PIPO 4 bit dùng D flip flop



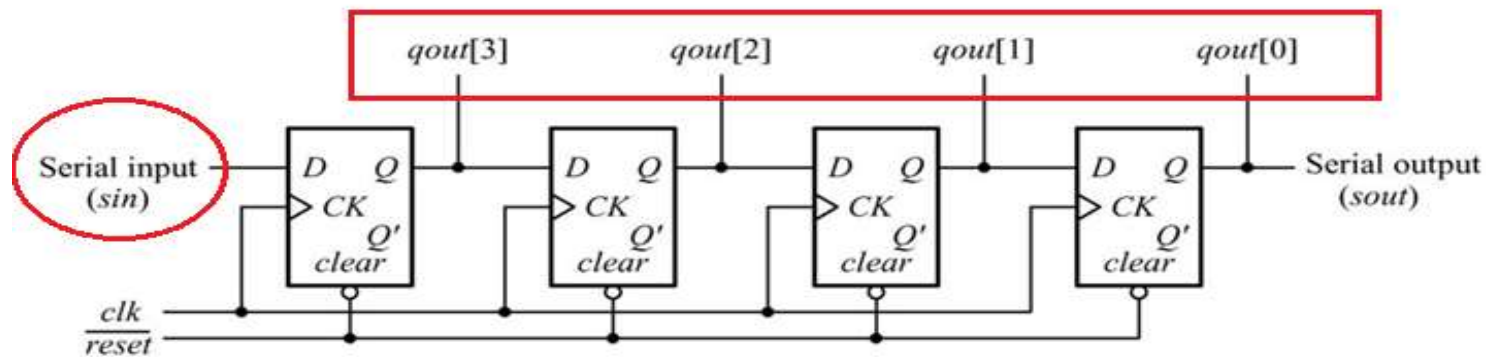
chương trình verilog mô tả thanh ghi PIPO 4 bit dùng D_FF, ck tác động theo cạnh lên.

```
module register (ck, din,qout);  
input ck;  
input [3:0] din;  
output reg [3:0]qout;  
always @ (posedge ck )  
Begin  
    qout <= din  
end  
endmodule
```

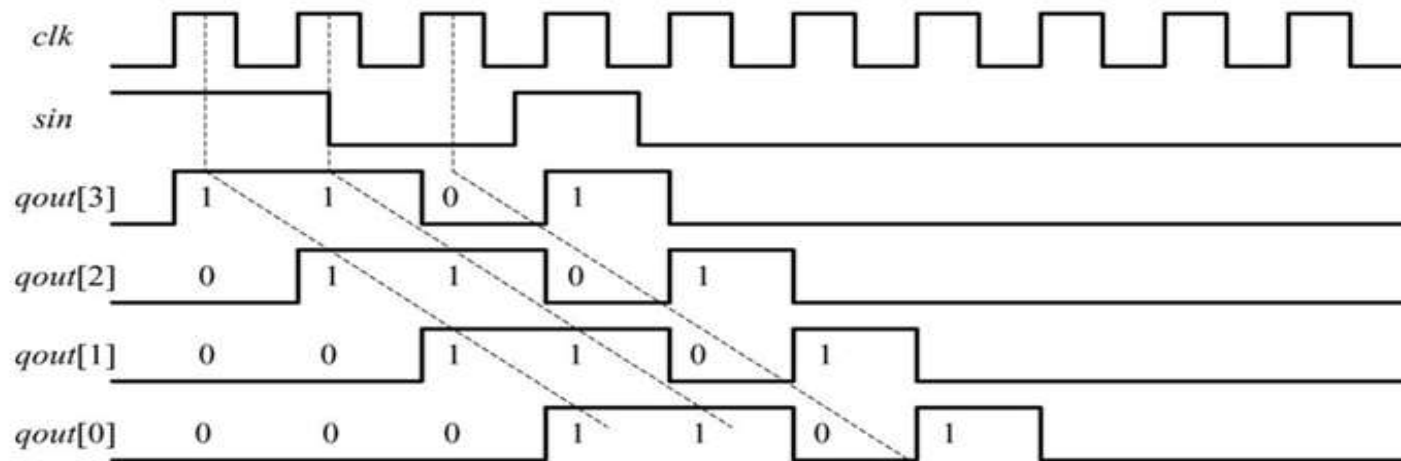


b. Thanh ghi dịch vào nối tiếp, ra song song (SIPO)

Ví dụ : viết chương trình verilog mô tả thanh ghi SIPO 4 bit dùng D flip flop



(a) Logic circuit



Gợi ý:

```
module shift_register(clk, reset_n, din, qout);  
Parameter N = 4; // number of bits  
....  
output reg [N-1:0] qout;  
  
always @(posedge clk or negedge reset_n)  
    if (!reset_n) qout <= {N{1'b0}};  
    else          qout <= {din, qout[N-1:1]};
```



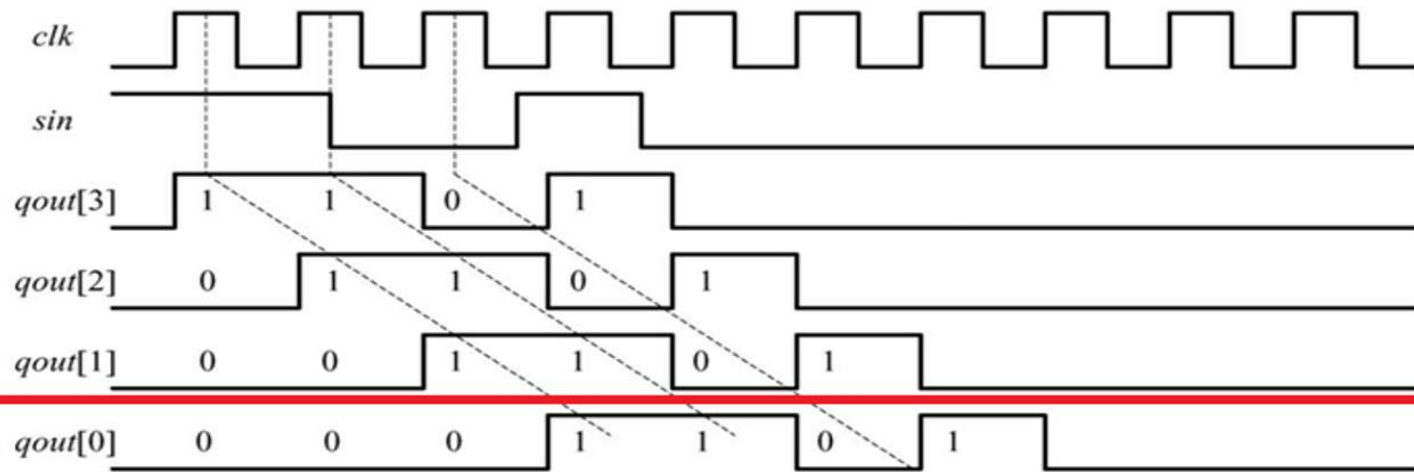
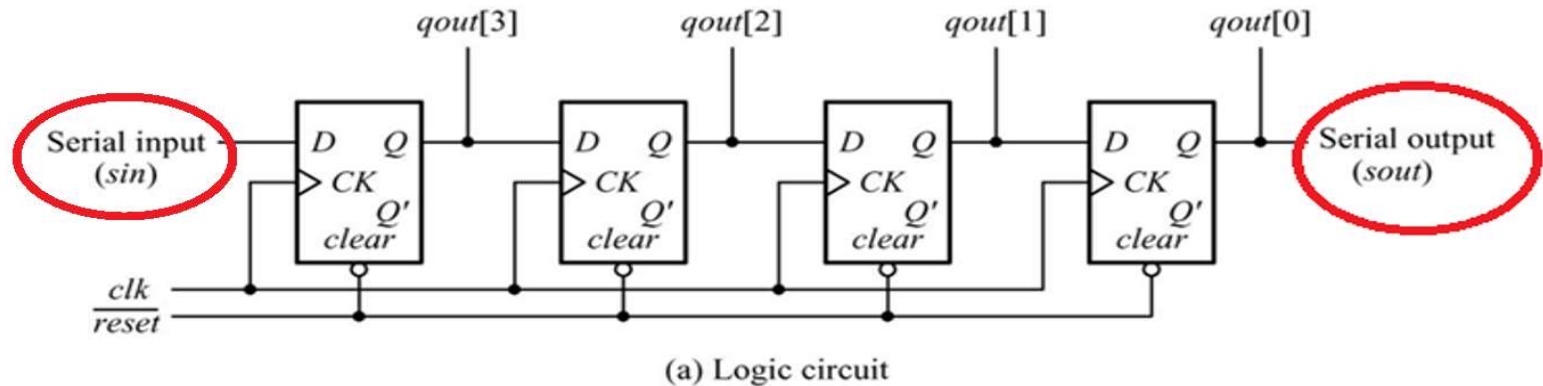
Ví dụ: viết code verilog mô tả thanh ghi dịch SIPO 4 bit. Chân ck tác động theo cạnh lên, chân rs không đồng bộ tích cực mức thấp

```
module shift_register_SIPO (ck, rs,din,qout);  
input ck,rs;  
input  din;  
output reg [3:0]qout;  
always @ (posedge ck or negedge rs )  
begin  
    if (!rs) qout <= 4'b0000;  
    else  qout <= {din,qout[3:1]};  
end  
endmodule
```



c. Thanh ghi dịch vào nối tiếp, ra nối tiếp (SISO)

Ví dụ : viết chương trình verilog mô tả thanh ghi SISO 4 bit dùng D flip flop



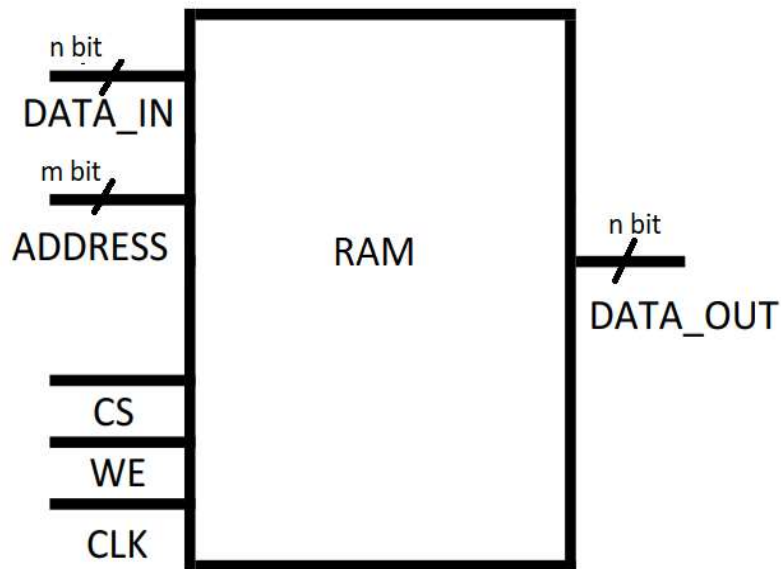
```
module shift_register_SISO (ck, rs,din,sout);  
input ck,rs;  
reg [3:0]q;  
input  din;  
output reg out;  
always @ (posedge ck or negedge rs )  
begin  
    if (!rs) begin  
        q <= 0;  sout <= 0;  
    end  
    else  begin  
        q <= q>>1;   q[3] <= din;  
    end  
end  
assign sout = q[0];  
endmodule
```



7. Thiết kế ROM-RAM

6.1 Thiết kế RAM (Random Access Memory)

a. Sơ đồ chân tổng quát



Các chân input:

- DATA_IN : n bit dữ liệu ngõ vào.
- ADDRESS : m bit mã hóa địa chỉ RAM, với m bit thì mã hóa được 2^m đường địa chỉ cho RAM
- CS : chân cho phép RAM hoạt động. Khi CS tích cực thì RAM hoạt động
- WE : chân cho phép RAM ghi/đọc dữ liệu.

Các chân output:

DATA_OUT : n bit dữ liệu ngõ ra (có số bit bằng số bit của DATA_IN)



b. Cách đọc thông số RAM

Cách viết chung: RAM $2^m \times n$

trong đó:

2^m : là số địa chỉ cho phép lưu dữ liệu của RAM

n : là số bit lưu dữ liệu của RAM

VD : RAM $256 \times 8 = \text{RAM } 2^8 \times 8$

Ta đọc được các thông số sau:

Dữ liệu lưu trữ trong ram được lưu vào 8 bit $\rightarrow$ DATA_IN
và DATA_OUT có 8 bit

Ram có 256 đường địa chỉ $\rightarrow$ mã hóa bằng 8 bit $\rightarrow$ chân
ADDRESS có 8 bit



c. Nguyên lý hoạt động

Cấp xung CLK cho RAM

Cho CS = 1 để RAM hoạt động

Cho chân WE = 1 : ghi dữ liệu cho RAM từ DATA_IN đến địa chỉ ADDRESS

Cho chân WE = 0 : đọc dữ liệu từ RAM tại địa chỉ ADDRESS đưa ra DATA_OUT



Ví dụ : Viết code verilog mô tả RAM 8x8

Giải:

Phân tích đề bài

RAM 8x8 có 8 địa chỉ → chân address có 3 bit

Dữ liệu 8 bit → chân datain và dataout có 8 bit

cs = 1 cho phép ram hoạt động

we = 1 : cho phép ghi dữ liệu

we = 0 : cho phép đọc dữ liệu



```
module RAM( dataIn, dataOut, Addr,CS,WE, Clk );
input  [7:0] dataIn;
output reg [7:0] dataOut;
input  [2:0] Addr;
input CS, WE, Clk;
reg [7:0] SRAM [7:0]; //SRAM co 8 dia chi , moi dia chi luu du lieu 8 bit
always @ (posedge Clk)
begin
    if (CS == 1'b1)
    begin
        if (WE == 1'b1) // ghi du lieu
            SRAM [Addr] <= dataIn;
        else // doc du lieu
            dataOut <= SRAM [Addr];
    end
    else
        dataOut <= 8'bxxxxxxxx;
    end
endmodule
```

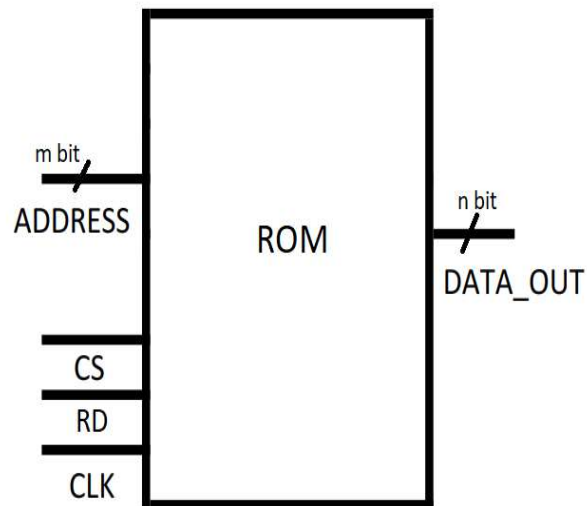


Kết quả mô phỏng



6.2 Thiết kế ROM (Read-only memory)

a. Sơ đồ chân tổng quát



Các chân input:

- ADDRESS : m bit mã hóa địa chỉ ROM, với m bit thì mã hóa được 2^m đường địa chỉ cho ROM
- CS : chân cho phép ROM hoạt động. Khi CS tích cực thì ROM hoạt động
- RD : chân cho phép ROM đọc dữ liệu.

Các chân output:

DATA_OUT : n bit dữ liệu ngõ ra



b. Cách đọc thông số ROM

Cách viết chung: ROM $2^m \times n$

trong đó: 2^m : là số địa chỉ cho phép lưu dữ liệu của ROM

n : là số bit lưu dữ liệu của ROM

VD : ROM $256 \times 8 = \text{ROM } 2^8 \times 8$

Ta đọc được các thông số sau:

Dữ liệu lưu trữ trong rom được lưu vào 8 bit $\rightarrow$ DATA_OUT
có 8 bit

Rom có 256 đường địa chỉ $\rightarrow$ mã hóa bằng 8 bit $\rightarrow$ chân
ADDRESS có 8 bit

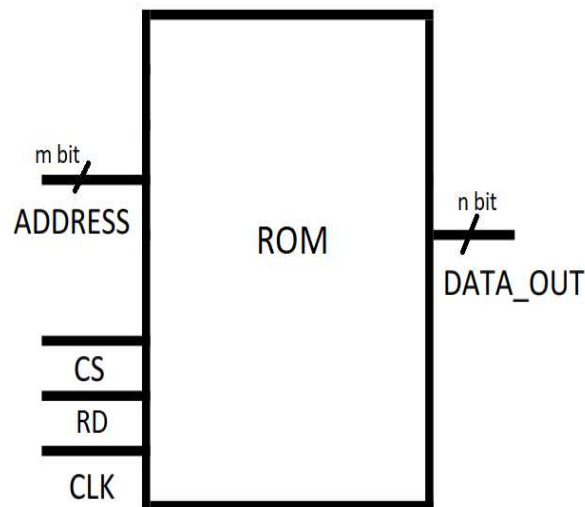


c. Nguyên lý hoạt động

Cấp xung CLK cho ROM

Cho CS = 1 để ROM hoạt động

Cho chân RD = 1 : cho phép đọc dữ liệu từ ROM tại địa chỉ ADDRESS đưa ra DATA_OUT



d. Ví dụ : viết code verilog mô tả ROM 16x4

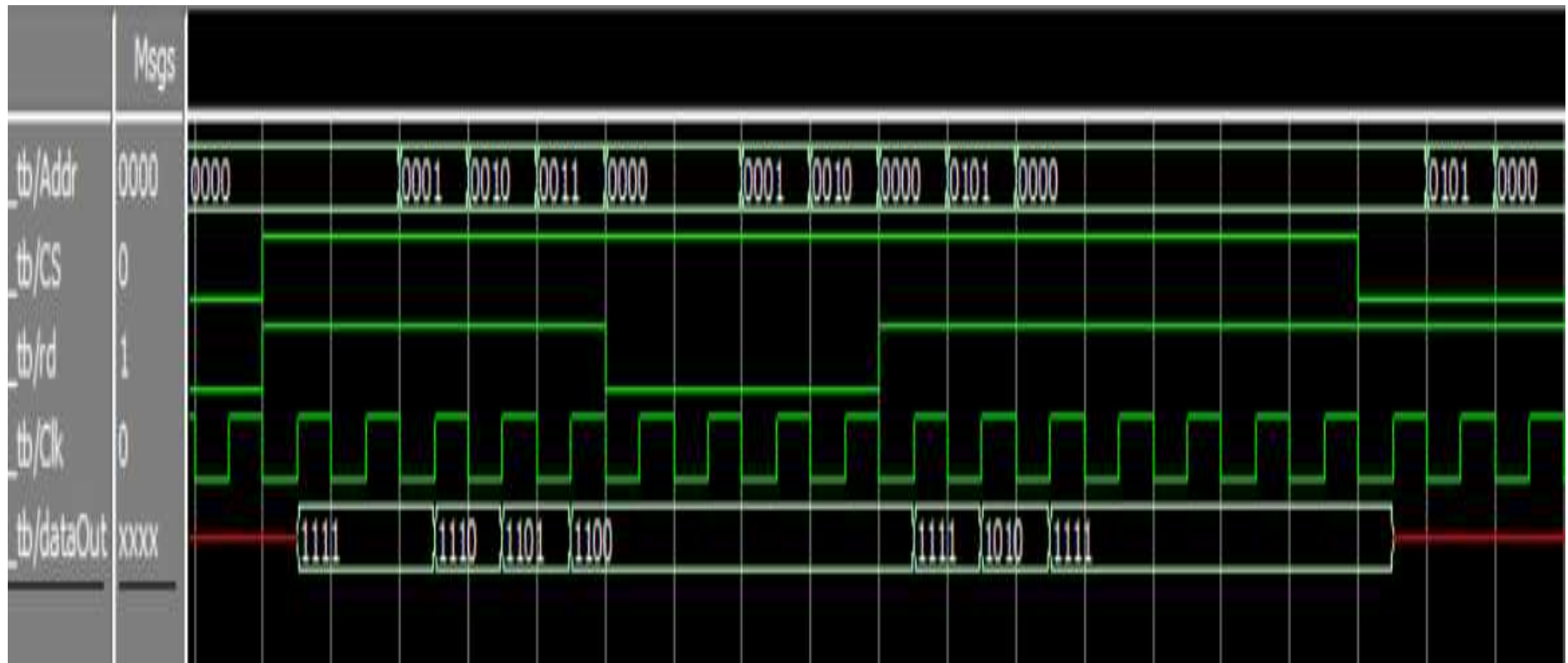
```
module ROM (dataOut, Addr, CS, rd, Clk );
output reg [3:0] dataOut;
input [3:0] Addr;
input CS,rd, clk;
reg [3:0] rom [15:0]; //ROM co 16 dia chi , moi dia chi
//luu du lieu 4 bit
initial
begin
rom[4'b0000] = 4'b1111;
rom[4'b0001] = 4'b1110;
rom[4'b0010] = 4'b1101;
rom[4'b0011] = 4'b1100;
rom[4'b0100] = 4'b1011;
rom[4'b0101] = 4'b1010;
rom[4'b0110] = 4'b1001;
rom[4'b0111] = 4'b1000;
rom[4'b1000] = 4'b0111;
rom[4'b1001] = 4'b0110;
rom[4'b1010] = 4'b0101;
rom[4'b1011] = 4'b0100;
rom[4'b1100] = 4'b0011;
rom[4'b1101] = 4'b0010;
rom[4'b1110] = 4'b0001;
rom[4'b1111] = 4'b0000;
end
```

```
// rom 16x4
// rom co 16 dia chi (4 chan addr ngo vao),
//du lieu 4 bit
// cs =1 : cho phep rom
// rd = 1 : cho phep doc du lieu

always @ (posedge Clk)
begin
if (CS == 1'b1)
begin
if (rd == 1'b1)
dataOut <= rom [Addr];
else
dataOut <= dataOut;
end
else
dataOut = 4'bxxxx;
end
Endmodule
```



Kết quả mô phỏng





**Thank you for
listening**

